

Evaluating TurboQuant-Family Compression on CLIP Multimodal Embeddings

CS 639: Introduction to Foundation Models — Project Proposal, Spring 2026

aagarwal235@wisc.edu akumar323@wisc.edu cajain@wisc.edu rsingh@wisc.edu
sagarwal@wisc.edu sbarjatya@wisc.edu sdatta9@wisc.edu

Overarching Idea & Motivation

Semantic image search at scale is a storage problem. CLIP [1] encodes images and text into a shared 512-dimensional embedding space, enabling retrieval across modalities by dot-product similarity. A billion-image index requires over 2 TB of float32 embeddings before any search infrastructure. Compressing those vectors is the obvious fix, but existing methods either require expensive dataset-specific training or sacrifice too much retrieval quality.

TurboQuant [2], presented at ICLR 2026, changes this. By combining PolarQuant [4] and QJL [3], it compresses vectors to 2.5–3.5 bits per dimension with negligible accuracy loss, no training, and compression that is instant regardless of database size. Its guarantees are information-theoretically grounded: it operates within a small constant factor ($\approx 2.7\times$) of the best any quantizer can achieve. The catch is that every evaluation in the paper uses text embeddings—specifically OpenAI embeddings on DBpedia and GloVe vectors. Whether the same guarantees hold for multimodal embeddings is untested.

This matters because CLIP embeddings are not just text. Image and text vectors occupy systematically offset regions of the shared sphere, a phenomenon known as the modality gap [5], and cross-modal retrieval depends on that offset surviving compression intact. TurboQuant’s random-rotation step was never designed with this structure in mind. Recent work [6] shows that the modality gap makes CLIP embeddings less robust to quantization noise—but only naive uniform quantization was tested. Whether a near-optimal quantizer like TurboQuant exhibits the same vulnerability is an open question that we address in this project.

Hypothesis

We predict that TurboQuant, QJL, and PolarQuant can compress CLIP embeddings to 3–4 bits with acceptable retrieval quality, but that cross-modal tasks (text-to-image, image-to-text) will degrade more sharply at low bit-widths (1–2 bits) than same-modal tasks (text-to-text, image-to-image). The reason is that cross-modal retrieval depends on the geometric alignment between image and text vectors, which we expect to be more sensitive to information loss than within-modality similarity. On the efficiency side, we expect QJL and TurboQuant to compress the database near-instantly, giving them a practical advantage over FAISS PQ which requires training.

Methodology

Datasets and Model

We use **Flickr30k** (30,000 images, 5 captions each) as our primary benchmark. All images and captions are encoded with frozen **CLIP ViT-B/32** into 512-dimensional embeddings, explicitly L2-normalized to unit norm (required by PolarQuant’s polar decomposition), in a single GPU session on Colab Pro. All subsequent experiments are CPU-only.

Compression Methods

TurboQuant has no official code, and the QJL and PolarQuant repos ship GPU kernels wired into LLM inference pipelines. We adapt the modular Python reference implementation from `scos-lab/turboquant` [8], which provides clean NumPy/SciPy modules for TurboQuantMSE, TurboQuantProd, rotation, scalar quantization, and QJL with 49 tests reproducing the paper’s distortion tables. Concretely, we build: **QJL** [3] using the asymmetric inner-product estimator, **PolarQuant** [4] using MSE-optimized scalar quantization with precomputed Lloyd-Max codebooks, and **TurboQuant** [2] as the full two-stage composition (Polar-

Quant + 1-bit QJL residual correction) for unbiased inner-product estimates. **FAISS Product Quantization** [7] serves as the trained baseline. The uncompressed float32 database is the gold standard.

Evaluation

For each method and bit-width in $\{1, 2, 3, 4, 8, 32\}$, we run four retrieval tasks and report Recall@1, @5, and @10: two cross-modal tasks (text→image, image→text) as the primary test, and two same-modal tasks (text→text, image→image) as controls. Queries stay at full float32; only the database is compressed. We report means and 95% confidence intervals over 5 runs with different random rotation seeds. Beyond recall, we measure memory (index size in MB), retrieval speed (mean query latency in ms), and compression time for each method.

References

-
- [1] A. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision,” *ICML*, 2021.
 - [2] A. Zandieh, M. Daliri, M. Hadian, V. Mirrokni, “TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate,” *ICLR*, 2026.
 - [3] A. Zandieh, M. Daliri, I. Han, “QJL: 1-Bit Quantized JL Transform for KV Cache Quantization with Zero Overhead,” *AAAI*, 2025.
 - [4] I. Han, P. Kacham, A. Karbasi, V. Mirrokni, A. Zandieh, “PolarQuant: Quantizing KV Caches with Polar Transformation,” *AISTATS*, 2026.
 - [5] W. Liang et al., “Mind the Gap: Understanding the Modality Gap in Multi-modal Contrastive Representation Learning,” *NeurIPS*, 2022.
 - [6] Y. Yaras et al., “Is the Modality Gap a Bug or a Feature? A Robustness Perspective,” arXiv:2603.29080, 2026.
 - [7] J. Johnson, M. Douze, H. Jégou, “Billion-scale Similarity Search with GPUs,” *IEEE Trans. Big Data*, 2019.
 - [8] scos-lab, “TurboQuant reference implementation,” <https://github.com/scos-lab/turboquant>, 2026.